

M17 / PROTECTED ANSWERS

M17 Explained Reviews

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first 3

DE17-01 | From requirements to architecture - worked recovery 3

DE17-02 | Batch vs streaming decisions - worked recovery 3

DE17-03 | Warehouse vs lakehouse - worked recovery 4

DE17-04 | Managed service vs build yourself - worked recovery 5

DE17-05 | Scaling and bottleneck reasoning - worked recovery 6

DE17-06 | Reliability and failure domains - worked recovery 6

DE17-07 | Cost-aware architecture - worked recovery 7

DE17-08 | Security and governance by design - worked recovery 8

DE17-09 | Promotion foundations: CDC, Contracts, Vault, Mesh, Kubernetes - worked recovery . . 8

DE17-10 | Architecture review and defense - worked recovery 9

Read this first

Open this book after saving your own attempt. Compare the reasoning and result, repair the cause of a mistake, then close the answer and try changed inputs.

Open only after a genuine attempt. Each review shows a defensible architecture answer, trade-off reasoning, common mistake, validation and a fresh retry.

DE17-01 | From requirements to architecture - worked recovery

Strong answer target. An architecture decision sheet linking each important requirement to a design response and a measurable proof. Another engineer should be able to disagree with your choice without first guessing your assumptions.

One acceptable worked answer. A retailer wants hourly store sales by 07:00 local time, can tolerate 30 minutes of delay during recovery, must keep cardholder data out of analytics, and has a two-person data team. Translate this into: batch ingestion every 15 minutes or hourly, curated sales grain, PII tokenization before analytics, a published freshness SLO, replayable raw evidence, simple managed components, and an alert if reconciliation or freshness fails. Notice that the requirements already rule out some unnecessarily complex designs.

Why that answer is defensible. The decision is anchored to a requirement and includes a failure/cost/ownership boundary. A different option can also be correct if the requirement set changes; the review is not asking you to memorize the same technology.

Common wrong answer. A team selects Kafka, Spark, Kubernetes and a lakehouse because these technologies appear in job descriptions. The workload is one 2 GB nightly file with next-morning reporting. The result creates 24x7 operating cost and a steep on-call burden without a requirement that needs streaming or cluster orchestration.

How to validate. Use the matching practice file or table. Check that every major requirement has a design response, every major state has recovery behavior, and each claim about scale/cost/security has an assumption or measurable control.

Reviewer challenge. A reviewer should be able to point to every box and ask which requirement forced it to exist. If the answer is only 'best practice', the design is not yet grounded. Separate hard constraints from preferences: residency law or RPO zero may be hard; a preferred cloud service may be negotiable. Also record revisit triggers, such as volume crossing 5x, latency tightening below one minute, or the team gaining platform support. Architecture is a current decision under stated conditions, not a permanent truth.

Another defensible option. Alternative: a warehouse-first design can be equally valid if the source is already structured, BI dominates, and raw/replay requirements are satisfied elsewhere. The review should compare complexity and recovery, not force a lakehouse.

Changed retry. Change the requirement from 'next morning' to 'fraud alert within 10 seconds' and explain exactly which architecture boundaries must change and which can remain batch.

DE17-02 | Batch vs streaming decisions - worked recovery

Strong answer target. A latency/value table and a justified choice that names state, replay, ordering and operational implications - not merely 'streaming is faster'.

One acceptable worked answer. Compare three workloads. Payroll: monthly, correctness dominates latency - batch. Card fraud: suspicious transaction must be acted on in seconds - streaming event path plus durable replay. Executive daily revenue: source events may arrive continuously, but a governed hourly/daily aggregate can remain batch or micro-batch. The same company can rationally use all three.

Why that answer is defensible. The decision is anchored to a requirement and includes a failure/cost/ownership boundary. A different option can also be correct if the requirement set changes; the review is not asking you to memorize the same technology.

Common wrong answer. A product team asks for 'real time' because the phrase sounds modern. No one can identify a user action that changes within an hour. The stream becomes a permanent on-call system while reports are opened once each morning.

How to validate. Use the matching practice file or table. Check that every major requirement has a design response, every major state has recovery behavior, and each claim about scale/cost/security has an assumption or measurable control.

Reviewer challenge. The hardest part is often not choosing batch or stream, but defining the consistency boundary between them. A streaming alert can be provisional while an hourly reconciliation becomes the accounting truth. That is acceptable when consumers know the contract. Conversely, two paths that silently compute the same metric differently create permanent reconciliation debt. If you design a hybrid path, name which system owns final truth, how late corrections flow, and how replay avoids double effects.

Another defensible option. Alternative: micro-batch every 30 seconds may meet a near-real-time dashboard with much simpler state than true event-by-event processing. Choose it when the business action tolerates that delay.

Changed retry. Take one batch choice and tighten its useful latency by 100x. Re-evaluate source, storage, compute and monitoring without automatically assuming every component must become streaming.

DE17-03 | Warehouse vs lakehouse - worked recovery

Strong answer target. A comparison matrix that connects consumers and data shapes to a storage/serving choice and states what will remain intentionally outside that platform.

One acceptable worked answer. A finance reporting team needs highly governed SQL, stable semantic models and hundreds of BI queries. A data science team needs raw/semi-structured clickstream and Spark. One defensible architecture lands durable source data in object storage/lakehouse tables, curates reusable silver data, and serves governed finance marts through a warehouse/SQL serving surface. The design explains the boundary instead of forcing one label everywhere.

Why that answer is defensible. The decision is anchored to a requirement and includes a failure/cost/ownership boundary. A different option can also be correct if the requirement set changes; the review is not asking you to memorize the same technology.

Common wrong answer. A team migrates all BI tables into a lake because storage is cheaper but does not plan compaction, catalog permissions, concurrency or semantic serving. File cost falls while analyst experience and operations degrade.

How to validate. Use the matching practice file or table. Check that every major requirement has a design response, every major state has recovery behavior, and each claim about scale/cost/security has an assumption or measurable control.

Reviewer challenge. Think in data products and workload boundaries rather than brand categories. Raw evidence may live cheaply in object storage while governed serving data uses an engine optimized for concurrent SQL. The same Delta/Iceberg-style table may be queried by multiple engines, yet each query engine still has capacity, security and cost behavior. Your decision should include who owns table maintenance, catalog policy, semantic definitions and performance expectations; storage format alone does not solve those responsibilities.

Another defensible option. Alternative: warehouse-only can be defensible for mostly structured governed SQL workloads; lakehouse-only can be defensible when open multi-engine data is central. Hybrid is not automatically superior because it adds boundaries to operate.

Changed retry. Remove Spark/ML from the scenario and make 95% of demand governed BI. Re-evaluate whether the lakehouse remains central or becomes a landing/archive component.

DE17-04 | Managed service vs build yourself - worked recovery

Strong answer target. A weighted scorecard with assumptions and an explicit owner for the operational responsibilities that remain after the choice.

One acceptable worked answer. A six-person analytics team needs orchestration. Self-hosting a scheduler on Kubernetes gives control but adds cluster/scheduler upgrades, HA, secrets, logging and on-call. A managed orchestration service may cost more per visible unit yet reduce total ownership. If the company already has a platform team operating Kubernetes safely, the balance may change.

Why that answer is defensible. The decision is anchored to a requirement and includes a failure/cost/ownership boundary. A different option can also be correct if the requirement set changes; the review is not asking you to memorize the same technology.

Common wrong answer. The team self-hosts because the open-source software is free. Six months later one engineer spends most of the week on upgrades and broken credentials. License cost was zero; total cost of ownership was not.

How to validate. Use the matching practice file or table. Check that every major requirement has a design response, every major state has recovery behavior, and each claim about scale/cost/security has an assumption or measurable control.

Reviewer challenge. A useful total-ownership review includes at least five hidden jobs: upgrades, security response, backups/recovery, observability/on-call, and capacity planning. Put a named team beside each job. If no team owns it, self-hosting has an unpriced risk. Managed services also need an exit strategy: identify portable data formats, export/replay boundaries and provider-specific features you would have to replace. This turns vague lock-in fear into a concrete migration-cost discussion.

Another defensible option. Alternative: self-hosting can win when the organization already operates the platform safely and needs controls or economics a managed product cannot provide. Reuse existing capability rather than counting it as free.

Changed retry. Change the organization to a regulated platform team with 30 experienced SREs and a hard requirement for custom network/security controls. Re-score rather than keeping your first answer.

DE17-05 | Scaling and bottleneck reasoning - worked recovery

Strong answer target. A bottleneck table with measured/assumed capacities and the smallest change that raises end-to-end capacity.

One acceptable worked answer. A pipeline must process 36 million events in one hour: 10,000 events/sec average, but design for 2x headroom. Ingest supports 50k/sec, Spark supports 30k/sec, sink supports 12k/sec. Adding Spark executors cannot meet a 20k/sec target because sink is bottleneck. Options are sink batching/partitioning, a different serving design, a longer window or reduced write amplification.

Why that answer is defensible. The decision is anchored to a requirement and includes a failure/cost/ownership boundary. A different option can also be correct if the requirement set changes; the review is not asking you to memorize the same technology.

Common wrong answer. A job runs slowly, so the team doubles compute. The source API is throttled at 500 requests/minute. Cloud cost doubles while duration barely changes.

How to validate. Use the matching practice file or table. Check that every major requirement has a design response, every major state has recovery behavior, and each claim about scale/cost/security has an assumption or measurable control.

Reviewer challenge. Capacity planning should use percentiles and bursts, not only daily averages. A pipeline averaging 5k events/sec can still fail when a 10-minute burst hits 40k/sec. Include retry traffic and failure-mode capacity: if losing one partition or worker makes the remaining system exceed saturation, there is no real headroom. Observe queue depth and lag because they reveal when arrival rate exceeds service rate even before users see missed deadlines.

Another defensible option. Alternative: reducing work can beat adding capacity. Pre-aggregation, predicate pushdown, smaller serialization, fewer copies or changed serving grain may raise effective throughput more cheaply than scaling compute.

Changed retry. Simulate losing one worker/partition domain and keep 25% headroom. Recalculate whether the design still meets the processing window.

DE17-06 | Reliability and failure domains - worked recovery

Strong answer target. A failure-domain table: failure, blast radius, detection, safe state, recovery action, RTO/RPO evidence.

One acceptable worked answer. A daily pipeline has RTO 2 hours and RPO 0 for accepted raw files. Raw objects are immutable and replicated, transforms are replayable, curated publish is versioned, and last-known-good output remains available if validation fails. Compute can be recreated. The checkpoint state is backed up and tied to durable output so progress never outruns data.

Why that answer is defensible. The decision is anchored to a requirement and includes a failure/cost/ownership boundary. A different option can also be correct if the requirement set

changes; the review is not asking you to memorize the same technology.

Common wrong answer. All services run across two zones, but credentials live in one manually maintained file and the only checkpoint is on one VM disk. The diagram looks redundant; actual recovery has single points of failure.

How to validate. Use the matching practice file or table. Check that every major requirement has a design response, every major state has recovery behavior, and each claim about scale/cost/security has an assumption or measurable control.

Reviewer challenge. Reliability also requires correlated-failure thinking. Two copies in the same account, credential boundary, region, or deployment can fail together. Ask 'what common thing can kill both?' for every redundancy claim. Recovery plans should be exercised using known-good evidence: restore a checkpoint copy, replay a bounded raw range, verify key counts and business totals, then republish. A backup whose restore path is untested is only a hope, not proven recovery.

Another defensible option. Alternative: for a non-critical internal report, last-known-good output plus next-day recovery may be better than expensive cross-region active-active. Reliability should match business impact.

Changed retry. Tighten RTO from 4 hours to 15 minutes while keeping RPO zero. Identify which design changes cost more and which risks cannot be removed merely by adding replicas.

DE17-07 | Cost-aware architecture - worked recovery

Strong answer target. A cost model showing assumptions, unit costs, volume, reliability overhead and a sensitivity case. Cost claims are traceable rather than 'serverless is cheaper'.

One acceptable worked answer. Option A is serverless query at 5 currency units/TB, scanning 40 TB/day = 200/day. Partitioning/pruning cuts scans to 8 TB/day = 40/day. Option B uses always-on cluster costing 120/day but handles predictable heavy transformations. Compare workload shape and operations, then model 30-day cost and sensitivity at 2x volume.

Why that answer is defensible. The decision is anchored to a requirement and includes a failure/cost/ownership boundary. A different option can also be correct if the requirement set changes; the review is not asking you to memorize the same technology.

Common wrong answer. To save money a team removes duplicate raw retention and keeps only transformed outputs. A later bug cannot be replayed or audited. The immediate storage saving destroys recoverability and creates much larger incident cost.

How to validate. Use the matching practice file or table. Check that every major requirement has a design response, every major state has recovery behavior, and each claim about scale/cost/security has an assumption or measurable control.

Reviewer challenge. Cost reviews should protect unit economics from misleading totals. A growing bill can be healthy if business volume grows faster and cost per million events falls; a flat bill can hide waste if workload shrinks. Attribute spend to workloads/environments, keep non-production schedules deliberate, and model data movement. Human operating cost matters too: a technically cheap self-hosted cluster that consumes half an engineer can be more expensive than a managed service with higher infrastructure charges.

Another defensible option. Alternative: a more expensive managed service may have lower TCO after on-call and engineering labor. Conversely predictable heavy workloads can make reserved/always-on capacity cheaper than per-query serverless.

Changed retry. Double data volume and cut query frequency by half. Recompute rather than assuming your original winner stays cheapest.

DE17-08 | Security and governance by design - worked recovery

Strong answer target. A security/governance matrix connecting persona -> data asset -> allowed action -> control -> evidence. No wildcard 'owner' role used as a shortcut.

One acceptable worked answer. Orders arrive with email and payment token. Ingest identity can write raw but not read curated. Transform identity reads raw and writes curated. Analytics users read masked curated tables; only an approved support role can access restricted contact data. Secrets are retrieved from a managed secrets system; audit logs and lineage record access/change paths.

Why that answer is defensible. The decision is anchored to a requirement and includes a failure/cost/ownership boundary. A different option can also be correct if the requirement set changes; the review is not asking you to memorize the same technology.

Common wrong answer. A pipeline fails with permission denied, so an engineer grants the service account subscription-owner access. The incident clears, but a small operational problem becomes a platform-wide security risk.

How to validate. Use the matching practice file or table. Check that every major requirement has a design response, every major state has recovery behavior, and each claim about scale/cost/security has an assumption or measurable control.

Reviewer challenge. Security architecture should include the failure path. During incidents, engineers often request broad emergency access. Define a controlled break-glass mechanism with approval, time limit and audit trail rather than normalizing permanent admin rights. Governance should also describe change: who can approve a schema change, who owns quality incidents, how sensitive classification propagates, and how deleted/retained data is proven. Security is an operating model as well as encryption settings.

Another defensible option. Alternative: centralizing a sensitive serving layer may simplify control for a small organization, while a large domain model may decentralize ownership with federated policies. The security properties must remain explicit either way.

Changed retry. Add a third-party data processor and cross-region copy. Identify new trust/compliance boundaries before changing service choices.

DE17-09 | Promotion foundations: CDC, Contracts, Vault, Mesh, Kubernetes - worked recovery

Strong answer target. A five-column map: technology/concept, problem solved, where it sits, major risk/ownership, and when not to use it. You can explain each in 60 seconds without pretending production mastery.

One acceptable worked answer. An e-commerce company has PostgreSQL orders, a lakehouse, and many domain teams. Debezium-like CDC publishes order changes to Kafka; a data contract

defines order-event keys, required fields and compatibility; workload identity/Vault supplies short-lived database credentials; the Orders domain owns the curated order data product; the platform team may run shared connectors/processors on managed Kubernetes. Each component has a different job. Removing one does not mean another automatically replaces it.

Why that answer is defensible. The decision is anchored to a requirement and includes a failure/cost/ownership boundary. A different option can also be correct if the requirement set changes; the review is not asking you to memorize the same technology.

Common wrong answer. A team says 'we are doing data mesh, so we need Kubernetes and Kafka.' That confuses organizational/data-product principles with implementation choices. Mesh can use many technologies; Kubernetes does not create domain ownership or trustworthy data products.

How to validate. Use the matching practice file or table. Check that every major requirement has a design response, every major state has recovery behavior, and each claim about scale/cost/security has an assumption or measurable control.

Reviewer challenge. Use the promotion map to avoid resume-driven architecture. CDC does not remove the need for idempotent consumers; contracts do not stop every semantic misunderstanding; Vault does not automatically make an overprivileged application safe; data mesh does not eliminate central platform/governance needs;

Kubernetes does not make a workload scalable by itself. Each concept shifts a specific problem and introduces its own ownership. Promotion readiness means you can discuss these boundaries clearly and know when deeper specialist study is required.

Another defensible option. Alternative: a small daily-file team may need none of CDC, mesh or Kubernetes, but can still benefit from a lightweight data contract and proper secrets handling. Promotion knowledge includes knowing what not to deploy.

Changed retry. Remove real-time requirements and move to daily file exchange with a small team. Re-evaluate which five concepts remain useful and which would be unjustified complexity.

DE17-10 | Architecture review and defense - worked recovery

Strong answer target. A one-page architecture brief, diagram, ADRs and 5-minute verbal defense where each major component has a requirement-based reason and a failure/recovery story.

One acceptable worked answer. Defend a regional retail lakehouse: hourly batch ingestion, immutable raw, Delta silver/gold, warehouse/SQL serving, managed orchestration, least-privilege identities, freshness/reconciliation gates, last-known-good publish, cost budgets and replay. Reviewers challenge: why not streaming? why not only warehouse? what if region fails? what if volume 5x? The defense returns to the requirement table and quantified trade-offs.

Why that answer is defensible. The decision is anchored to a requirement and includes a failure/cost/ownership boundary. A different option can also be correct if the requirement set changes; the review is not asking you to memorize the same technology.

Common wrong answer. The presenter answers every challenge by naming another service. The diagram grows but requirements, costs and ownership remain unclear. Complexity is being used to hide uncertainty instead of resolving it.

How to validate. Use the matching practice file or table. Check that every major requirement has a design response, every major state has recovery behavior, and each claim about scale/cost/security has an assumption or measurable control.

Reviewer challenge. A mature review explicitly records rejected options. If you choose managed batch, state why streaming or self-hosting lost under the present requirements. Then define a trigger that would reopen the decision. This protects the team from cargo-cult architecture and from treating an old ADR as eternal. In interviews, saying 'I would validate peak rate and consumer latency before finalizing partition count' is stronger than inventing a confident number with no evidence.

Another defensible option. Alternative: two architectures can both pass review when they meet requirements with different cost/operations trade-offs. A strong defense explains why the chosen trade-off fits this organization now.

Changed retry. Use the fresh Gate-B-style case with different latency, budget and compliance constraints. Defend a changed architecture without copying your first diagram.